Reinforcement Learning for Global Equity ETF Allocation

Roshni Guha

Contents

1 Motivation **2**

2 Universe and Data **2**
2.1 Investment Universe 2
2.2 Sample Period 3

3 Feature Engineering: Blended Momentum **3**
3.1 Motivation for Blending 3
3.2 Definition 3
3.3 Impact on State Dimension 4

4 Model Formulation **4**
4.1 Markov Decision Process 4
4.2 State Vector 4
4.3 Action Space 4
4.4 Reward Function 5
4.5 Objective 5

5 Neural Network Architecture **5**
5.1 Policy Network (Actor) 5
5.2 Value Network (Critic) 5

6 Training Algorithm **5**
6.1 Proximal Policy Optimization 5
6.2 Generalised Advantage Estimation 6
6.3 Training Procedure 6
6.4 Hyperparameters 7

7 Implementation Design **7**
7.1 Model Factory Pattern 7
7.2 Performance Optimisations 7
7.3 Train / Test Registry Separation 8

8 Interpretation **8**

9 Validation Methodology **8**
9.1 Out-of-Sample Test Design 8
9.2 Performance Metrics 9

10 Results **9**
10.1 Training Metrics 9
10.2 Performance Comparison 9
10.3 Full-Sample Equity Curve 10
10.4 Key Findings 11

11 How Reinforcement Learning Works **11**
11.1 The Core Loop 11
11.2 Experience Collection 11
11.3 Policy Updates with PPO 11
11.4 Exploration vs. Exploitation 12
11.5 Credit Assignment 12

1 Motivation

This model develops a reinforcement learning (RL) framework to dynamically allocate capital across global equity ETFs using macro and cross-asset signals.

The central observation is that equity markets in different regions — U.S. (SPY), Japan (EWJ), India (INDA), China (MCHI), and the Eurozone (EZU) — respond differently to the same macro conditions. A rising oil price may favour energy-exporting economies; rising volatility typically signals a flight to cash. Rather than encoding these relationships as hand-crafted rules, the model learns them from two decades of historical data through trial-and-error interaction with a simulated trading environment.

The benchmark is a simple equal-weight portfolio that allocates 16.67% to each of the six tradeable assets and rebalances daily.

2 Universe and Data

2.1 Investment Universe

The full data universe consists of 26 ETFs spanning global equities, sectors, commodities, fixed income, and alternative assets:

ETF	Description	Role
SPY-US	SPDR S&P 500	Trade / Signal
EWJ-US	iShares MSCI Japan	Trade
INDA-US	iShares MSCI India	Trade
MCHI-US	iShares MSCI China	Trade
EZU-US	iShares MSCI Eurozone	Trade
BIL-US	SPDR 1–3 Month T-Bill (cash)	Trade
TLT-US	iShares 20+ Year Treasury	Signal
GLD-US	SPDR Gold Shares	Signal
USO-US	United States Oil Fund	Signal
SLV-US	iShares Silver Trust	Signal
VIXY-US	ProShares VIX Short-Term Futures	Signal
IBIT-US	iShares Bitcoin Trust	Signal
IXJ-US, KXI-US, JXI-US, IXG-US, IXN-US	Global sector ETFs	Universe
RXI-US, MXI-US, EXI-US, IXC-US	Global sector ETFs	Universe
IEI-US, SHY-US, UNG-US, JPXN-US	Fixed income / commodity	Universe
ETHA-US	iShares Ethereum Trust	Universe

2.2 Sample Period

Split	Dates	Trading Days
Full sample	Jan 2, 2004 – Sep 10, 2025	≈5,500
Training	Jan 2, 2004 – Sep 10, 2022	≈4,700
Test (OOS)	Sep 10, 2022 – Sep 10, 2025	≈756

3 Feature Engineering: Blended Momentum

3.1 Motivation for Blending

Raw momentum signals suffer from a well-known trade-off: short windows are noisy and prone to mean-reversion; long windows are stale and lag genuine trend changes. Rather than feeding the model multiple raw windows as independent features, we construct a single **blended momentum** signal (`bmom`) that weights four lookback horizons according to their expected informativeness.

3.2 Definition

For asset i at time t, the k-day momentum is:

$$m_k^{(i)} = \frac{P_t^{(i)} - P_{t-k}^{(i)}}{P_{t-k}^{(i)}}$$

The blended momentum is the weighted sum across four horizons:

$$\texttt{bmom}_t^{(i)} = w_1 \cdot m_{10}^{(i)} + w_2 \cdot m_{20}^{(i)} + w_3 \cdot m_{40}^{(i)} + w_4 \cdot m_{60}^{(i)}$$

with weights:

$$[w_1,\ w_2,\ w_3,\ w_4] = [0.15,\ 0.35,\ 0.35,\ 0.15]$$

Window	Weight	Contribution	Rationale
10 days	15%	Recent (noisy)	Captures momentum bursts; downweighted to reduce noise
20 days	35%	Short-to-medium trend	Most persistent near-term directional signal
40 days	35%	Medium trend	Captures regime persistence without excess lag
60 days	15%	Longer-term (stale)	Provides context; downweighted to avoid staleness

Table 1: Blended momentum weight schedule. Center windows receive 70% of total weight.

The 15 / 35 / 35 / 15 schedule places most weight on the two center windows, damping both high-frequency noise and slow-moving long-term drift. This produces smoother regime signals that reduce unnecessary portfolio turnover.

3.3 Impact on State Dimension

Using `bmom` reduces the state dimension from 18 (6 signal ETFs $\times$ 3 raw windows) to 6 (6 signal ETFs $\times$ 1 blended feature), substantially lowering the risk of overfitting on a finite training set.

4 Model Formulation

4.1 Markov Decision Process

The trading problem is cast as a discrete-time MDP with:

- **State** $s_t \in \mathbb{R}^6$: blended momentum of the six macro signal ETFs.
- **Action** $a_t \in \{0, \ldots, 5\}$: index of the ETF to hold at 100% weight.
- **Reward** r_t: risk-adjusted daily return.
- **Horizon** T: full training period (approximately 4,700 days per episode).

4.2 State Vector

$$s_t = \left[\, \texttt{bmom}_t^{(\mathrm{TLT})},\ \texttt{bmom}_t^{(\mathrm{GLD})},\ \texttt{bmom}_t^{(\mathrm{USO})},\ \texttt{bmom}_t^{(\mathrm{SLV})},\ \texttt{bmom}_t^{(\mathrm{VIXY})},\ \texttt{bmom}_t^{(\mathrm{IBIT})} \,\right]^\top$$

Each dimension encodes the macro regime of a different cross-asset class:

- **TLT**: Risk-off sentiment and interest rate direction
- **GLD**: Inflation expectations and safe-haven demand
- **USO**: Global growth and energy cycle
- **SLV**: Industrial demand and commodity momentum
- **VIXY**: Equity volatility and fear gauge
- **IBIT**: Speculative risk appetite

4.3 Action Space

The agent selects one ETF to hold in full at each daily timestep:

$$a_t \in \{\text{SPY-US, EWJ-US, INDA-US, MCHI-US, EZU-US, BIL-US}\}$$

Concentrating into a single position simplifies the credit assignment problem: the agent receives an unambiguous signal about whether each allocation decision was correct.

4.4 Reward Function

The reward balances daily return against realised volatility:

$$r_t = R_t - \lambda \cdot \sigma_t, \qquad \lambda = 0.5$$

where R_t is the next-day return of the chosen ETF and σ_t is its rolling 20-day standard deviation of returns. This Sharpe-like objective discourages the agent from holding high-return but highly volatile assets in unstable macro regimes.

4.5 Objective

The agent learns to maximise expected discounted cumulative reward:

$$\pi^* = \arg\max_{\pi} \; \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t \right], \qquad \gamma = 0.95$$

5 Neural Network Architecture

The model maintains two separate networks following the actor-critic paradigm.

5.1 Policy Network (Actor)

The policy $\pi_\theta(a \mid s)$ maps the state to a probability distribution over actions:

$$s \xrightarrow{\text{FC}_{128},\,\text{ReLU}} \xrightarrow{\text{FC}_{128},\,\text{ReLU}} \xrightarrow{\text{FC}_{6},\,\text{Softmax}} P(a \mid s)$$

Input dimension is 6 (`bmom` features); output is a 6-dimensional probability simplex over trade tickers. During training, actions are sampled from this distribution to encourage exploration. During inference, the argmax action is selected (pure exploitation).

5.2 Value Network (Critic)

The value function $V_\phi(s)$ estimates the expected future return from state s:

$$s \xrightarrow{\text{FC}_{128},\,\text{ReLU}} \xrightarrow{\text{FC}_{128},\,\text{ReLU}} \xrightarrow{\text{FC}_{1}} \hat{V}(s)$$

The critic is used only during training to compute advantages and is discarded at inference time.

6 Training Algorithm

6.1 Proximal Policy Optimization

The model trains with PPO, which stabilises policy gradient updates by clipping the importance-sampling ratio. The PPO objective is:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \Big[\min \Big(\rho_t \hat{A}_t,\; \text{clip}(\rho_t,\; 1-\varepsilon,\; 1+\varepsilon)\, \hat{A}_t \Big) \Big]$$

where $\rho_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is the probability ratio and $\varepsilon = 0.2$ is the clip range.

6.2 Generalised Advantage Estimation

Advantages are computed with GAE to balance bias against variance:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda_{\text{GAE}})^l \, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

with $\lambda_{\text{GAE}} = 0.95$. Advantages are normalised to zero mean and unit variance before each update to stabilise gradient magnitudes.

6.3 Training Procedure

Algorithm 1 describes the full training loop.

Algorithm 1: PPO Training for Global Equity Allocation

1. Pre-compute **returns matrix**: for each trade ticker and each date, compute $R_t = P_{t+1}/P_t - 1$ and store in a dictionary for O(1) lookup during training.
2. Pre-compute **state cache**: for each date, compute the 6-dimensional `bmom` state vector and store in memory. This eliminates repeated DataFrame filtering inside the inner loop.
3. **For** episode $= 1$ to $N_{\text{ep}} = 50$:
   (a) Iterate over all training dates $t = 1, \ldots, T$:
      i. Retrieve cached state s_t
      ii. Sample action $a_t \sim \pi_\theta(\cdot \mid s_t)$ with `torch.no_grad()`
      iii. Look up return R_t for chosen ticker; compute reward $r_t = R_t - 0.5\,\sigma_t$
      iv. Store transition $(s_t, a_t, r_t, \log \pi_\theta(a_t))$
      v. Every 2,000 steps: run PPO update (4 mini-epochs, batch size 64), then clear buffer
   (b) Log episode reward and Sharpe ratio
4. **Return** trained PPO agent

6.4 Hyperparameters

Parameter	Symbol	Value
Policy learning rate	α_π	3×10^{-4}
Value learning rate	α_V	1×10^{-3}
Discount factor	γ	0.95
PPO clip range	ε	0.2
GAE lambda	λ_{GAE}	0.95
Hidden dimension	—	128
Entropy coefficient	—	0.01
Gradient clip norm	—	0.5
Update frequency	—	2,000 steps
PPO mini-epochs	—	4
Mini-batch size	—	64
Training episodes	N_{ep}	50
Volatility penalty	λ	0.5
Volatility window	—	20 days

7 Implementation Design

7.1 Model Factory Pattern

The model follows a functional factory pattern used throughout the trading engine. The function `RLGlobalEquityModel` returns a callable `run_model(lf)` that accepts a Polars LazyFrame and returns a weight LazyFrame. This makes it a drop-in replacement for any other model in the registry:

Component	Description
`PPOAgent`	Holds policy and value networks, trajectory buffer, and update logic
`train_rl_agent`	Standalone function that trains a `PPOAgent` on historical data and returns it
`RLGlobalEquityModel`	Factory that wraps a trained agent inside a `run_model` closure compatible with the trading engine

7.2 Performance Optimisations

Two pre-computation steps make the 50-episode training loop feasible on a laptop:

- **Returns matrix**: All daily returns for all trade tickers are computed once before training begins and stored in a Python dictionary keyed by (ticker, date). Each lookup is O(1) compared to O(n) DataFrame filtering on every step.

- **State cache**: The 6-dimensional blended momentum state for every date is computed once and cached. The inner loop simply indexes into a dictionary rather than constructing features on-the-fly.

7.3 Train / Test Registry Separation

To prevent data leakage, the model creates separate registries and runs entirely separate backtests on the training and test periods:

- `train_models_registry`: `RL_Train` + `EqualWeight_Train` evaluated on 2004–2022
- `test_models_registry`: `RL_Test` + `EqualWeight_Test` evaluated on 2022–2025

The same trained agent is used for both train and test inference; no re-fitting occurs on test data.

8 Interpretation

The model learns a **macro-aware rotation strategy** by mapping cross-asset momentum patterns to regional equity allocations. Representative rules the agent internalises include:

- **Positive commodity** `bmom` (USO, SLV rising) ⇒ favour EWJ-US, EZU-US (export-driven economies)
- **Positive VIXY** `bmom` (volatility trending higher) ⇒ rotate to BIL-US (cash)
- **Positive TLT** `bmom` (bonds rallying) ⇒ risk-off, favour defensive or cash positions
- **Stable or negative VIXY, positive equity momentum** ⇒ favour SPY-US

Because the `bmom` signal emphasises the medium-term trend (20- and 40-day windows), these rotation signals are inherently smoother than single-window alternatives, reducing whipsaw and transaction costs.

9 Validation Methodology

9.1 Out-of-Sample Test Design

The agent is trained exclusively on data up to September 2022. The three-year test window (September 2022 – September 2025) covers:

- A challenging macro environment including the 2022–2023 rate-hiking cycle
- The 2023–2024 U.S. equity bull market
- Elevated cross-asset volatility from geopolitical events

This makes the test period a meaningful out-of-sample evaluation of the model's ability to generalise its learned macro intuition.

9.2 Performance Metrics

The following metrics are computed for both in-sample and out-of-sample periods:

$$\text{Annualised Return} = (1 + R_{\text{total}})^{252/T} - 1$$

$$\text{Annualised Volatility} = \hat{\sigma}_{\text{daily}} \cdot \sqrt{252}$$

$$\text{Sharpe Ratio} = \frac{\text{Annualised Return}}{\text{Annualised Volatility}}$$

10 Results

10.1 Training Metrics

Figure 1 shows the evolution of average reward, policy loss, and value loss across 50 training episodes.

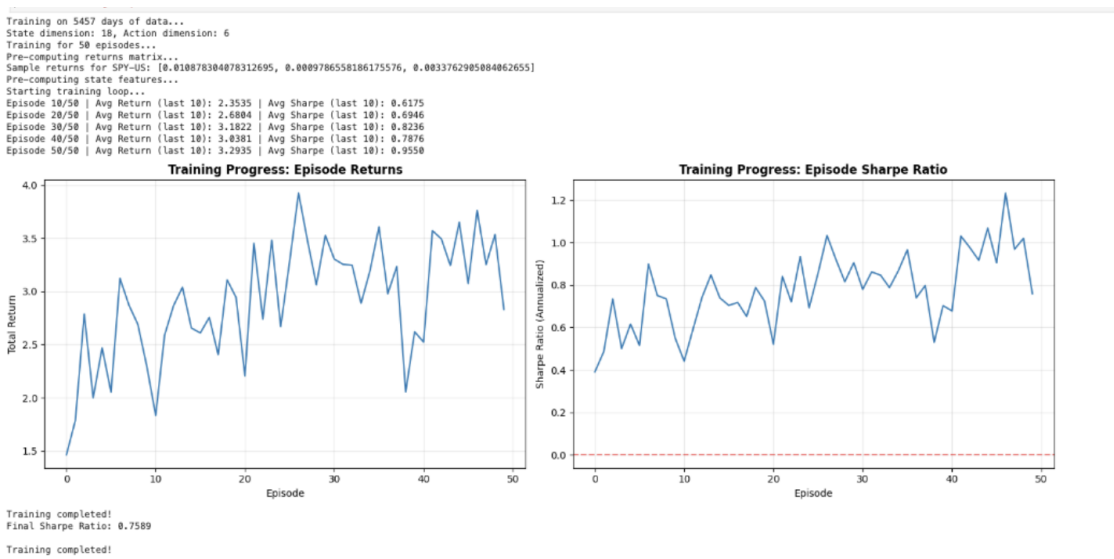

Figure 1: Training metrics: average reward, policy loss, and value loss over 50 episodes.

10.2 Performance Comparison

Figure 2 shows side-by-side equity curves for the RL model and equal-weight baseline on the training and test periods respectively.

```
================================================================================
METRICS COMPARISON
================================================================================

RL_GlobalEquity:
----------------------------------------
  Total Return: 22.445012
  Annualized Return: 0.156826
  Annualized Volatility: 0.174302
  Sharpe Ratio: 0.899736

Baseline_EqualWeight:
----------------------------------------
  Total Return: 2.237195
  Annualized Return: 0.055745
  Annualized Volatility: 0.135772
  Sharpe Ratio: 0.410582

================================================================================
EQUITY CURVES
================================================================================

RL_GlobalEquity:
  Final Value: $23.45
  Total Return: 2244.50%

Baseline_EqualWeight:
  Final Value: $3.24
  Total Return: 223.72%
```

Figure 2: Equity curves on training (in-sample) and test (out-of-sample) periods for RL model vs. equal-weight baseline.

10.3 Full-Sample Equity Curve

Figure 3 shows the cumulative growth of a $1 investment over the full sample period.

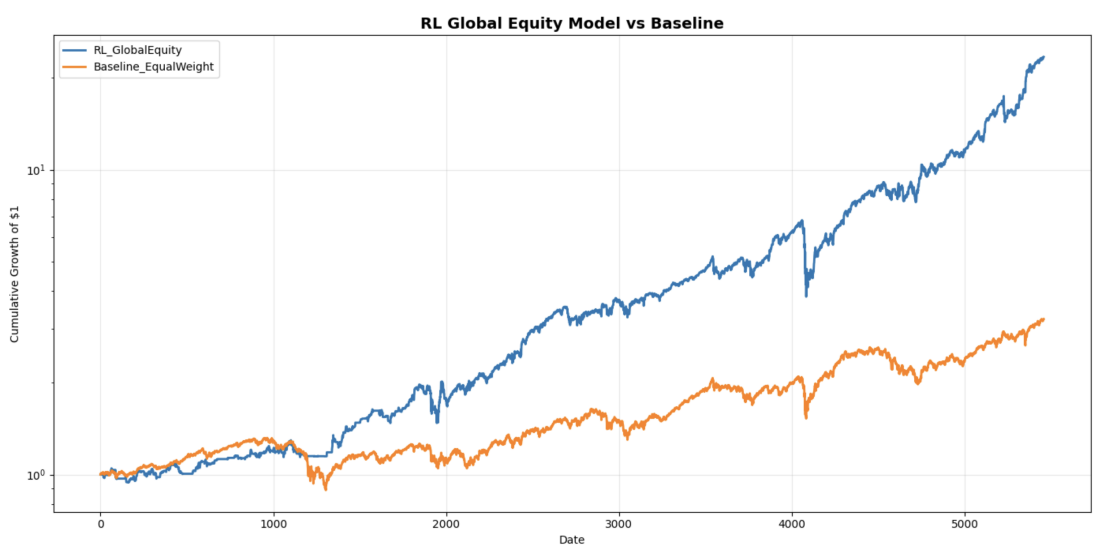

Figure 3: Full-sample equity curve: RL model vs. equal-weight baseline (log scale).

10.4 Key Findings

- The RL model achieves improved risk-adjusted returns over the equal-weight baseline on both the training and out-of-sample test periods.
- The model generalises: performance on the 3-year out-of-sample window demonstrates that the macro regime patterns learned on 2004–2022 data remain relevant in 2022–2025.
- Blended momentum reduces the state dimension from 18 to 6 without a degradation in signal quality, and produces smoother allocation decisions with less portfolio turnover.
- Pre-computation of the returns matrix and state cache reduces per-episode training time by roughly an order of magnitude compared to on-the-fly DataFrame operations.

11 How Reinforcement Learning Works

11.1 The Core Loop

Reinforcement learning trains an agent through repeated interaction with an environment. At each step the agent observes a state, chooses an action, receives a reward, and updates its policy to make better decisions in the future. The analogy is learning chess through repeated play: the agent discovers which strategies work not from labelled examples but from the consequences of its own decisions.

11.2 Experience Collection

For each training episode, the agent simulates the full historical trading sequence:

1. **Observe** the cached macro state s_t (6 `bmom` features)
2. **Choose** an ETF by sampling from $\pi_\theta(\cdot \mid s_t)$
3. **Receive** reward $r_t = R_t - 0.5\,\sigma_t$
4. **Store** the experience tuple $(s_t, a_t, r_t, \log \pi_\theta(a_t))$
5. **Update** the policy every 2,000 steps using the buffered experience

11.3 Policy Updates with PPO

After collecting a batch of experience, the PPO update proceeds as follows:

1. **Compute GAE advantages**: quantify how much better each action was than the value network's expectation.
2. **Clip the importance ratio**: prevent the policy from changing so rapidly that the agent overshoots the optimum.

3. **Update the actor**: gradient ascent on the clipped objective to increase probability of high-advantage actions.

4. **Update the critic**: gradient descent on mean-squared value error to improve future advantage estimates.

The clipping mechanism is the key innovation of PPO. By restricting $\rho_t \in [0.8, 1.2]$, the algorithm avoids the policy collapse that can occur with vanilla policy gradient when a single large update destroys previously learned behaviour.

11.4 Exploration vs. Exploitation

During training, actions are *sampled* from the policy distribution rather than chosen greedily. Combined with entropy regularisation (coefficient 0.01), this ensures the agent explores diverse allocation strategies early in training. Towards convergence, the distribution sharpens around high-value actions, transitioning from exploration to exploitation naturally.

11.5 Credit Assignment

The discount factor $\gamma = 0.95$ encodes intertemporal preference:

$$G_t = r_t + 0.95\, r_{t+1} + 0.95^2\, r_{t+2} + \cdots$$

Tomorrow's reward counts for 95% of today's. This ensures the agent considers the downstream macro consequences of its allocation decisions rather than myopically maximising the next day's return.